

Graph Neural Networks

Part of these slides are based on: M. Bronstein, Geometric Deep Learning Lecture Notes 7-8 (2024)

Darrick Lee

University of Edinburgh

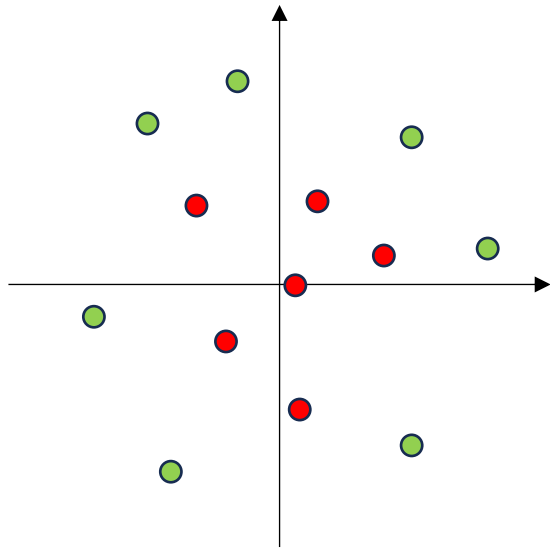
AGQ Computing Course

March 27, 2026

Recap: Tasks in ML

Classification

Input Space $\mathcal{X}$



Each point $x_i \in \mathcal{X}$ has a **label**
 $y_i \in \{\text{red}, \text{green}\}$



Classification

Find a function

$$f = (f_1, f_2) : \mathcal{X} \rightarrow \mathbb{R}^2$$

such that

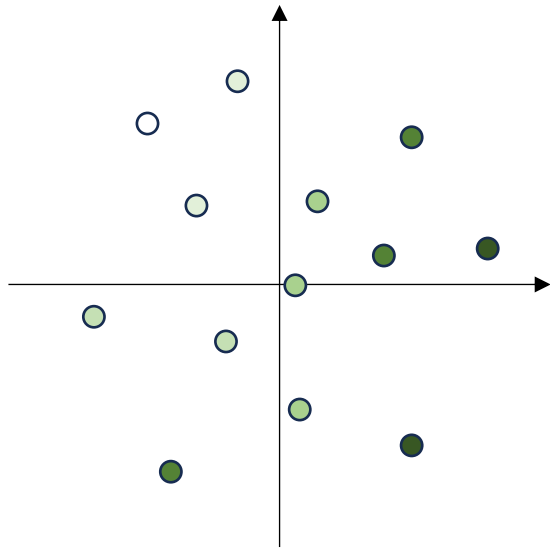
$$f_1(x) > f_2(x) \text{ } x \text{ is red}$$

$$f_2(x) > f_1(x) \text{ } x \text{ is green}$$

Recap: Tasks in ML

Regression

Input Space $\mathcal{X}$



Each point $x_i \in \mathcal{X}$ has a real-valued **label**

$$y_i \in \mathbb{R}$$

Regression

Find a function

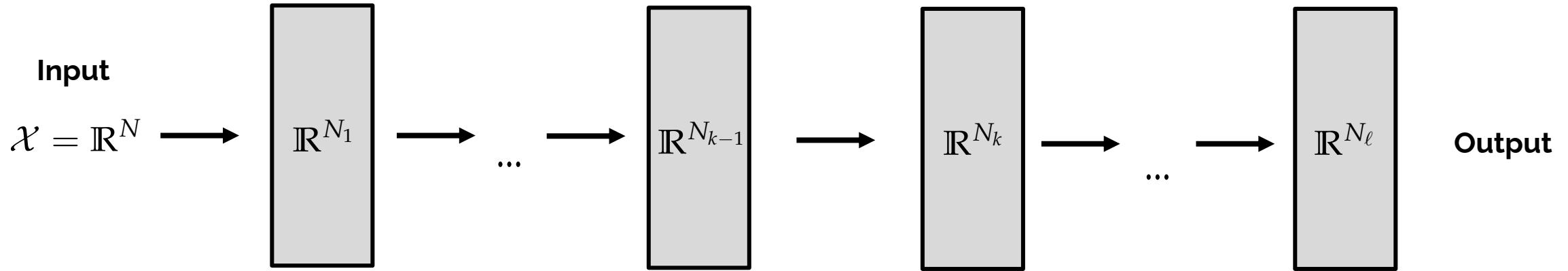
$$f : \mathcal{X} \rightarrow \mathbb{R}$$

such that

$$f(x_i) = y_i$$

Recap: Neural Networks

Neural networks are large parametrized families of functions to optimize over.



Trainable Parameters

$W_k \in L(\mathbb{R}^{N_{k-1}}, \mathbb{R}^{N_k})$ weights (represented as a matrix)

$b_k \in \mathbb{R}^{N_k}$ bias (represented as a vector)

All parameters

$$\theta = (W_k, b_k)_{k=1}^\ell \in \prod_{k=1}^\ell (L(\mathbb{R}^{N_{k-1}}, \mathbb{R}^{N_k}) \times \mathbb{R}^{N_k})$$

Neural Network Layer

$$\mathbf{x}^{(k)} = \sigma \left(W_k \mathbf{x}^{(k-1)} + b_k \right) \in \mathbb{R}^{N_k}$$

Neural Network

$$f_\theta : \mathcal{X} \rightarrow \mathbb{R}^\ell$$

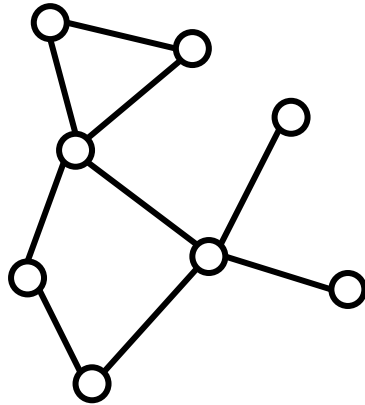
$$f_\theta(x) = x^{(\ell)}$$

Graph Data

So far: $\mathcal{X}$ = vectors, images

Today: $\mathcal{X}$ = graphs

Data are often not independent: they are equipped with relations that we wish to use.



Examples:

Social Networks

- Nodes: individuals
- Edges: friends / follows, etc.

Molecular Graphs

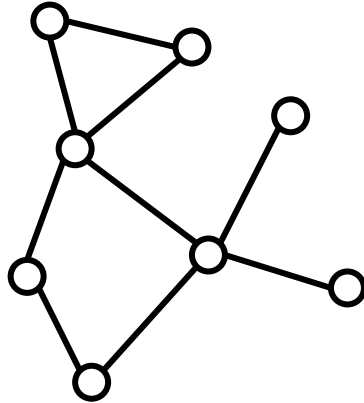
- Nodes: atoms
- Edges: chemical bonds

Cayley Graphs

- Nodes: group elements
- Edges: generators

Tasks using Graph Data

What can we learn about graphs?



Graph Classification/Regression

- Predicting properties of molecules

Node Classification/Regression

- What is the topic of a paper in a citation network
- Anomalous actors in communications/financial networks

Edge Classification/Regression

- Classify bond type in molecules
- Predict travel time / flow rate along a road in traffic networks

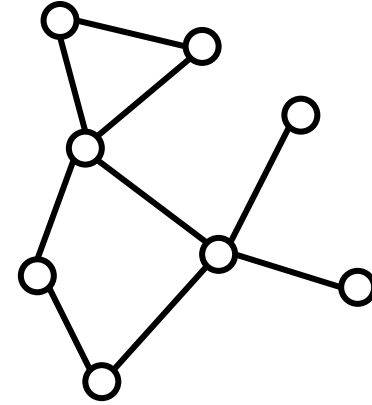
Graphs

A **graph** consists of:

a set of nodes $\mathcal{V} = \{1, \dots, n\}$

a set of edges $\mathcal{E} = \{(s_i, t_i)\}_{i=1}^E \subset \mathcal{V}^2$

two nodes are **adjacent**, $i \sim j$, if $(i, j) \in \mathcal{E}$



In ML applications, graphs may also come with:

node features $X = (X_1, \dots, X_n) \in \mathcal{A}^n$

node labels $Y = (Y_1, \dots, Y_n) \in \mathcal{B}^n$

edge features $F = (F_1, \dots, F_E) \in \mathcal{C}^E$

edge labels $H = (H_1, \dots, H_E) \in \mathcal{D}^E$

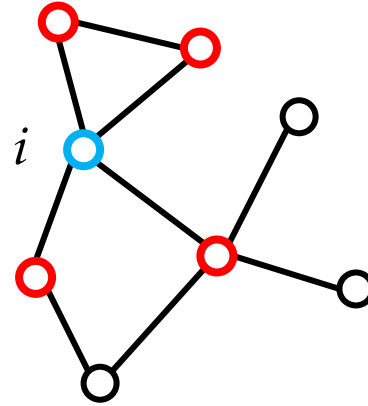
graph labels $R \in \mathcal{Q}$

Features (Attributes): Input quantities

Labels: Prediction target

Here, $\mathcal{A}, \mathcal{B}, \mathcal{C}, \mathcal{D}, \mathcal{Q}$ are just arbitrary sets.
Exactly what they are will depend on the dataset.

Neighbourhoods and Degree



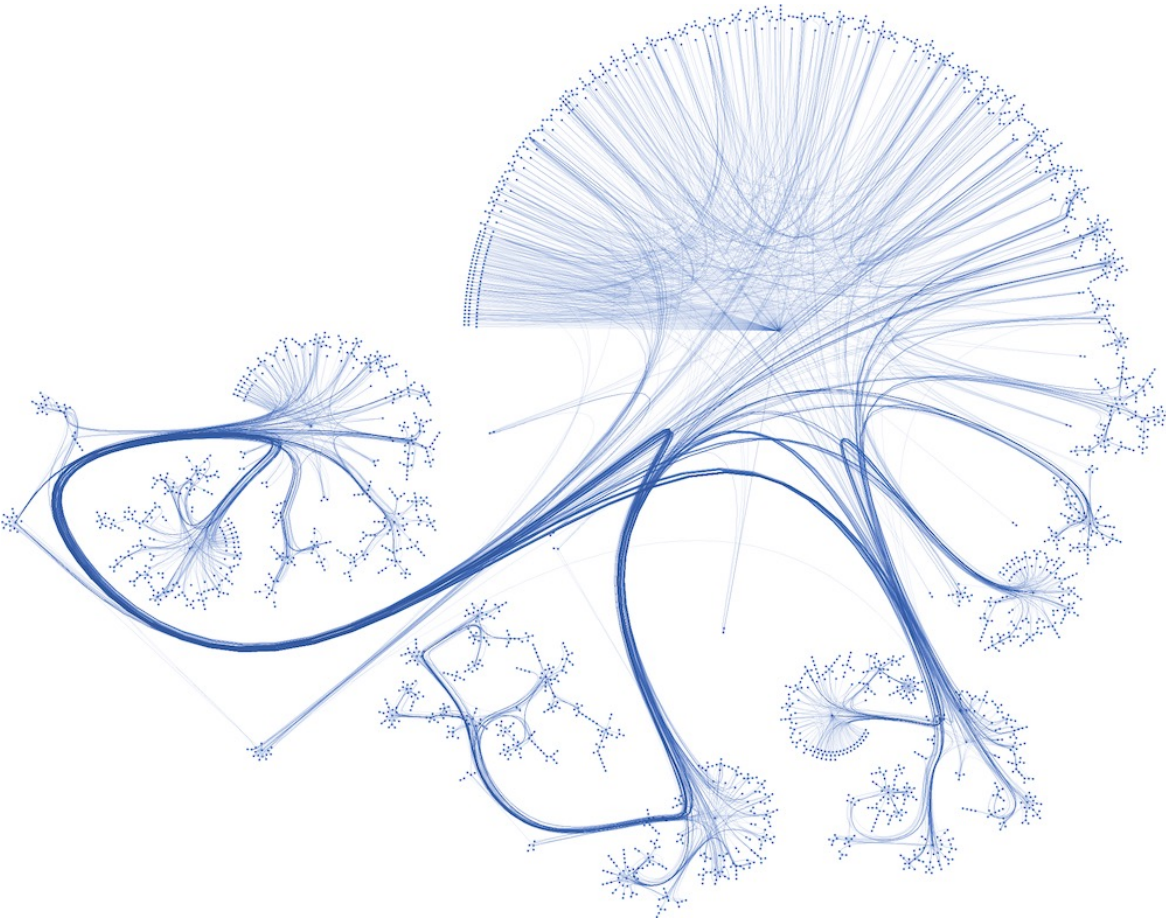
The **neighbourhood set** of a node is the set of nodes it is adjacent to.

$$\mathcal{N}_i := \{j \in \mathcal{V} : i \sim j\}$$

The **degree** of a node is the number of nodes it is adjacent to.

$$\deg(i) := |\mathcal{N}_i|$$

Example: Cora Dataset



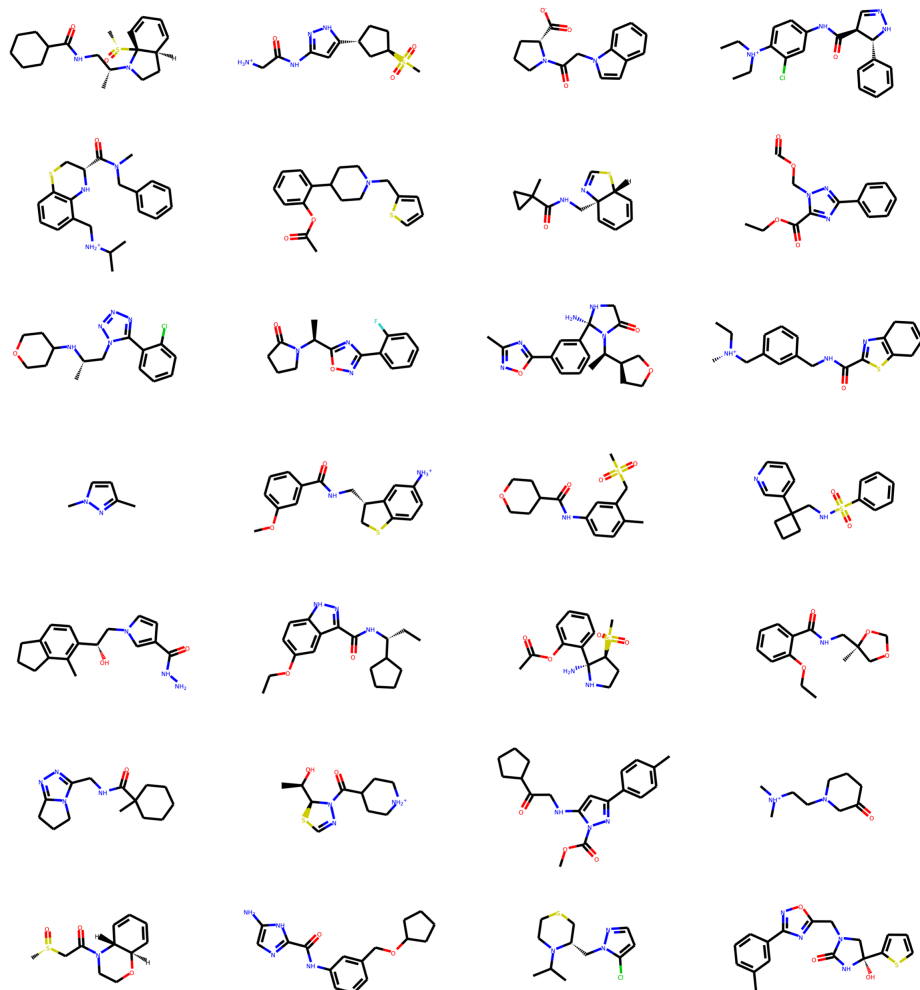
Visualization of Cora Dataset from: <https://graphsandnetworks.com/the-cora-dataset/>

Task: Node Classification

The **Cora Dataset** is a citation dataset consisting of:

- A single graph of 2,708 nodes and 5,429 edges
- Nodes are papers, edges are citations
- The **node features** are 1,433-dimensional
 - These are given by a **bag-of-words**
 - Given a dictionary of 1,433 words, the feature records the presence (1) or absence (0) of each word in the paper
 - $X \in \mathbb{R}^{2708 \times 1433}$
- The **node labels** is a classification into 7 ML topics:
 1. Case based
 2. Genetic Algorithms
 3. Neural Networks
 4. Probabilistic Methods
 5. Reinforcement Learning
 6. Rule Learning
 7. Theory
- There are no edge features/labels and no graph labels.

Example: ZINC Dataset



The **ZINC Dataset** is a molecular dataset consisting of:

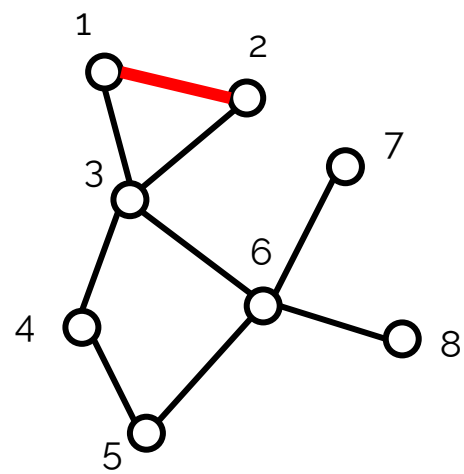
- 12,000 graphs representing molecules (nodes are atoms, edges are bonds)
- mean nodes = ~23.2 and mean edges = ~49.8
- **Node features:** integer representing atom type
- **Edge features:** integer representing bond type (single, double, triple, aromatic)
- No node labels or edge labels
- **Graph Label:** A real valued number representing a molecular property (constrained solubility)

Task: Graph Regression

Figure from:
Jin et al., Junction Tree Variational Autoencoder for Molecular Graph Generation, ICML (2018)

Graphs as Matrices

Graph



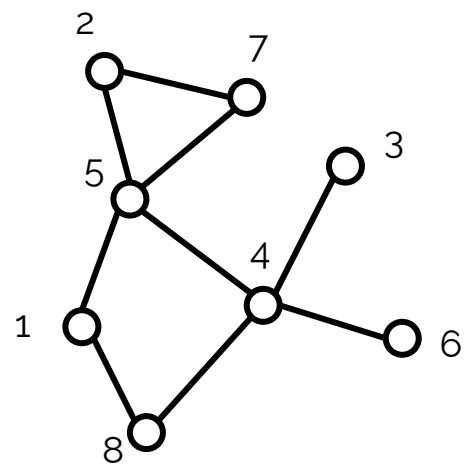
Adjacency Matrix

	1	2	3	4	5	6	7	8
1								
2								
3								
4								
5								
6								
7								
8								

Edge Indices

1	2	1	3	2	3	3	4	3
2	1	3	1	3	2	4	3	6

6	4	5	5	6	6	7	6	8
3	5	4	6	5	7	6	8	6

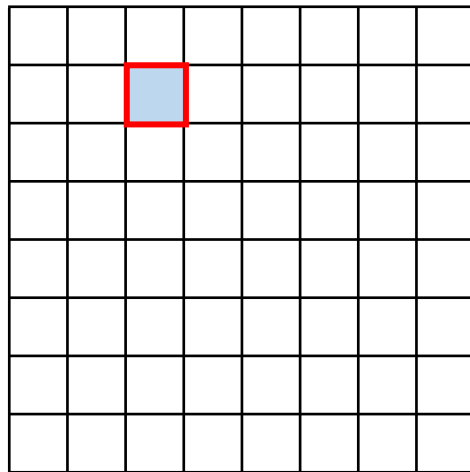
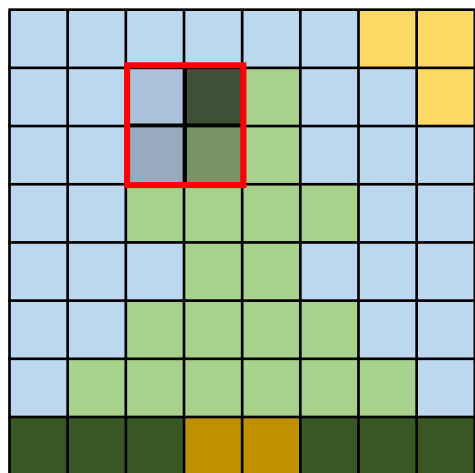


	1	2	3	4	5	6	7	8
1								
2								
3								
4								
5								
6								
7								
8								

1	5	1	8	2	5	2	7	3
5	1	8	1	5	2	7	2	4

4	4	5	4	6	4	8	5	7
3	5	4	6	4	8	4	7	5

Convolutional vs Graph Neural Networks



Images

$$\mathcal{X}_\ell = \{X : \mathbb{Z}^2 \rightarrow \mathbb{R}^\ell\}$$

Convolutional Layer

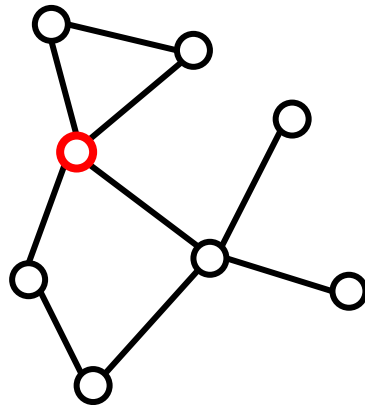
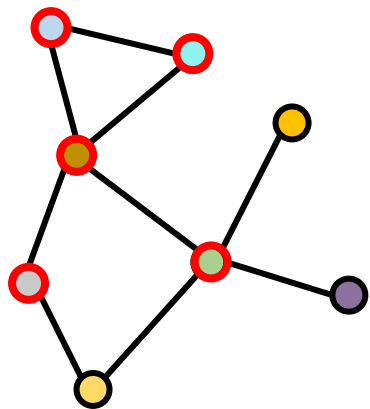
$$f : \mathcal{X}_{\ell_1} \rightarrow \mathcal{X}_{\ell_2}$$

Aggregate information from neighbouring pixels.

This is **translation-equivariant**:

$$\sigma \circ f = f \circ \sigma$$

$$\sigma \in \mathbb{Z}^2$$



Graphs

$$\mathcal{X}_\ell = \{X : G \rightarrow \mathbb{R}^\ell\}$$

Convolutional Layer

$$f : \mathcal{X}_{\ell_1} \rightarrow \mathcal{X}_{\ell_2}$$

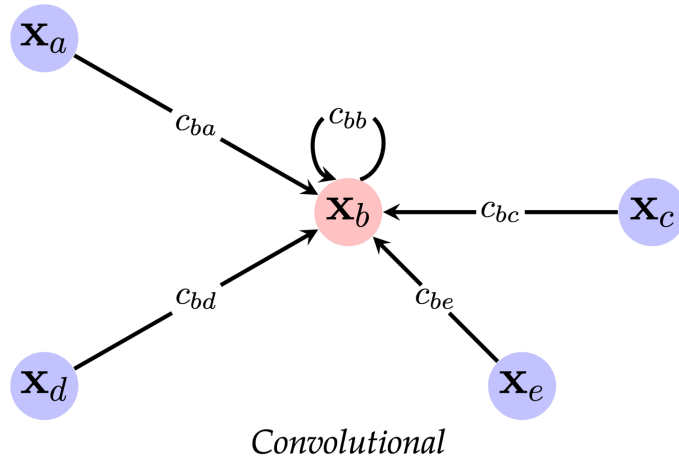
Aggregate information from neighbouring nodes.

This is **permutation-equivariant**:

$$\sigma \circ f = f \circ \sigma$$

$$\sigma \in \Sigma_n$$

Graph Convolutional Networks (GCN)



$$\phi \left(x_i, \sum_{j \in \mathcal{N}_i} c_{ij} \psi(x_j) \right)$$

$\mathcal{N}_i = \{j \in \mathcal{V} : i \sim j\}$ Neighbours of node i

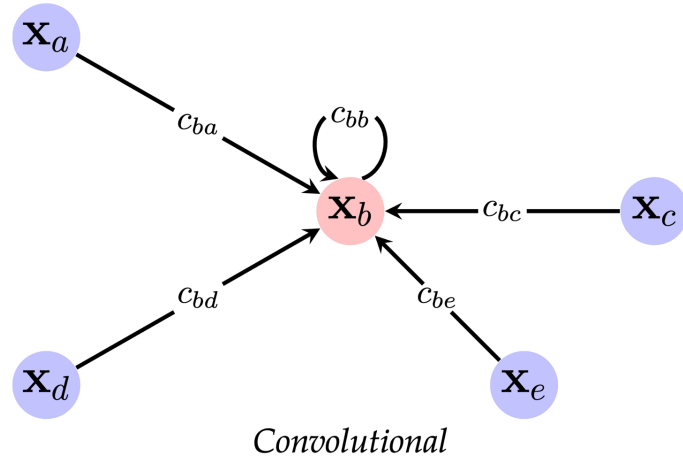
$\psi : \mathbb{R}^{\ell_1} \rightarrow \mathbb{R}^{\ell_2}$ Transformation

$c_{i,j}$ Weights from adjacency matrix

$\phi : \mathbb{R}^{\ell_1} \times \mathbb{R}^{\ell_2} \rightarrow \mathbb{R}^{\ell_2}$ Transformation

1. For each edge, compute a **message** from source node to target node
2. For each node, **aggregate** message from its neighbours
3. For each node, **update** the feature

Graph Convolutional Networks (GCN)



$$\phi \left(x_i, \sum_{j \in \mathcal{N}_i} c_{ij} \psi(x_j) \right)$$

A specific GCN architecture (which we'll be implementing):

First, apply a learned linear transformation

$$\psi \in L(\mathbb{R}^{\ell_1}, \mathbb{R}^{\ell_2}) \quad \psi(\mathbf{x}) = W\mathbf{x}$$

Aggregation weighted by degree normalization
(including a self-edge)

$$c_{ij} = \frac{1}{\sqrt{\deg(i) \deg(j)}}$$

Apply a bias and nonlinear activation function

$$\mathbf{x}_i \mapsto \sigma \left(\sum_{j \in \mathcal{N}_i} \frac{1}{\sqrt{\deg(i) \deg(j)}} W\mathbf{x}_j + b \right)$$

Trainable Parameters

$$W \in L(\mathbb{R}^{\ell_1}, \mathbb{R}^{\ell_2}) \quad b \in \mathbb{R}^{\ell_2}$$

GCN in Pytorch Geometric

We will use **Pytorch Geometric**, a library built on top of Pytorch to work with GNNs.

```
class GCNConv(MessagePassing):
    def __init__(self, in_channels, out_channels):
        super().__init__(aggr='add') # "add" aggregation (sum over neighbours)
        self.lin = nn.Linear(in_channels, out_channels, bias=False)
        self.bias = nn.Parameter(torch.zeros(out_channels))

    def forward(self, x, edge_index):
        # Step 1: Add self-loops to the edge index.
        edge_index, _ = add_self_loops(edge_index, num_nodes=x.size(0))

        # Step 2: Linearly transform the node features.
        # x = ...

        # Step 3: Compute the normalisation.
        # EXERCISE: Construct a size (E) array `norm` such that
        #   norm[i] = 1/sqrt(deg(edge_index[i,0]) * deg(edge_index[i,1])).
        norm = ... # This should be a 1D array of the same length as edge_index

        # Step 4: Call propagate. This calls self.message() on each edge,
        #   then aggregates (sums) messages at each target node.
        out = self.propagate(edge_index, x=x, norm=norm)

        return out + self.bias

    def message(self, x_j, norm):
        # EXERCISE: return the normalised message
        # (multiply each row of x_j by the corresponding entry of norm)
```

Constructor

Forward pass: same idea as other NNs
(run an input through the layer)

Message: For each edge, compute the message
(this is used in the forward pass)

GCN in Pytorch Geometric

We will use **Pytorch Geometric**, a library built on top of Pytorch to work with GNNs.

```
class GCNConv(MessagePassing):
    def __init__(self, in_channels, out_channels):
        super().__init__(aggr='add') # "add" aggregation (sum over neighbours)
        self.lin = nn.Linear(in_channels, out_channels, bias=False)
        self.bias = nn.Parameter(torch.zeros(out_channels))

    def forward(self, x, edge_index):
        # Step 1: Add self-loops to the edge index.
        edge_index, _ = add_self_loops(edge_index, num_nodes=x.size(0))

        # Step 2: Linearly transform the node features.
        # x = ...

        # Step 3: Compute the normalisation.
        # EXERCISE: Construct a size (E) array `norm` such that
        #   norm[i] = 1/sqrt(deg(edge_index[i,0]) * deg(edge_index[i,1])).
        norm = ... # This should be a 1D array of the same length as edge_index

        # Step 4: Call propagate. This calls self.message() on each edge,
        #   then aggregates (sums) messages at each target node.
        out = self.propagate(edge_index, x=x, norm=norm)

        return out + self.bias

    def message(self, x_j, norm):
        # EXERCISE: return the normalised message
        # (multiply each row of x_j by the corresponding entry of norm)
```

Specify how aggregation works: we will always use 'add'

GCN in Pytorch Geometric

We will use **Pytorch Geometric**, a library built on top of Pytorch to work with GNNs.

```
class GCNConv(MessagePassing):
    def __init__(self, in_channels, out_channels):
        super().__init__(aggr='add') # "add" aggregation (sum over neighbours)
        self.lin = nn.Linear(in_channels, out_channels, bias=False)
        self.bias = nn.Parameter(torch.zeros(out_channels))

    def forward(self, x, edge_index):
        # Step 1: Add self-loops to the edge index.
        edge_index, _ = add_self_loops(edge_index, num_nodes=x.size(0))

        # Step 2: Linearly transform the node features.
        # x = ...

        # Step 3: Compute the normalisation.
        # EXERCISE: Construct a size (E) array `norm` such that
        #   norm[i] = 1/sqrt(deg(edge_index[i,0]) * deg(edge_index[i,1])).
        norm = ... # This should be a 1D array of the same length as edge_index

        # Step 4: Call propagate. This calls self.message() on each edge,
        #   then aggregates (sums) messages at each target node.
        out = self.propagate(edge_index, x=x, norm=norm)

        return out + self.bias

    def message(self, x_j, norm):
        # EXERCISE: return the normalised message
        # (multiply each row of x_j by the corresponding entry of norm)
```

Initialize trainable parameters
 W, b

GCN in Pytorch Geometric

We will use **Pytorch Geometric**, a library built on top of Pytorch to work with GNNs.

```
class GCNConv(MessagePassing):
    def __init__(self, in_channels, out_channels):
        super().__init__(aggr='add') # "add" aggregation (sum over neighbours)
        self.lin = nn.Linear(in_channels, out_channels, bias=False)
        self.bias = nn.Parameter(torch.zeros(out_channels))

    def forward(self, x, edge_index):
        # Step 1: Add self-loops to the edge index.
        edge_index, _ = add_self_loops(edge_index, num_nodes=x.size(0))

        # Step 2: Linearly transform the node features.
        # x = ...

        # Step 3: Compute the normalisation.
        # EXERCISE: Construct a size (E) array `norm` such that
        #   norm[i] = 1/sqrt(deg(edge_index[i,0]) * deg(edge_index[i,1])).
        norm = ... # This should be a 1D array of the same length as edge_index

        # Step 4: Call propagate. This calls self.message() on each edge,
        #   then aggregates (sums) messages at each target node.
        out = self.propagate(edge_index, x=x, norm=norm)

        return out + self.bias

    def message(self, x_j, norm):
        # EXERCISE: return the normalised message
        # (multiply each row of x_j by the corresponding entry of norm)
```

Each node is treated as a neighbour
in our aggregation step:
Add this in as an edge

GCN in Pytorch Geometric

We will use **Pytorch Geometric**, a library built on top of Pytorch to work with GNNs.

```
class GCNConv(MessagePassing):
    def __init__(self, in_channels, out_channels):
        super().__init__(aggr='add') # "add" aggregation (sum over neighbours)
        self.lin = nn.Linear(in_channels, out_channels, bias=False)
        self.bias = nn.Parameter(torch.zeros(out_channels))

    def forward(self, x, edge_index):
        # Step 1: Add self-loops to the edge index.
        edge_index, _ = add_self_loops(edge_index, num_nodes=x.size(0))

        # Step 2: Linearly transform the node features.
        # x = ...

        # Step 3: Compute the normalisation.
        # EXERCISE: Construct a size (E) array `norm` such that
        #   norm[i] = 1/sqrt(deg(edge_index[i,0]) * deg(edge_index[i,1])).
        norm = ... # This should be a 1D array of the same length as edge_index

        # Step 4: Call propagate. This calls self.message() on each edge,
        #   then aggregates (sums) messages at each target node.
        out = self.propagate(edge_index, x=x, norm=norm)

        return out + self.bias

    def message(self, x_j, norm):
        # EXERCISE: return the normalised message
        # (multiply each row of x_j by the corresponding entry of norm)
```

Apply the linear transformation

$$Wx$$

GCN in Pytorch Geometric

We will use **Pytorch Geometric**, a library built on top of Pytorch to work with GNNs.

```
class GCNConv(MessagePassing):
    def __init__(self, in_channels, out_channels):
        super().__init__(aggr='add') # "add" aggregation (sum over neighbours)
        self.lin = nn.Linear(in_channels, out_channels, bias=False)
        self.bias = nn.Parameter(torch.zeros(out_channels))

    def forward(self, x, edge_index):
        # Step 1: Add self-loops to the edge index.
        edge_index, _ = add_self_loops(edge_index, num_nodes=x.size(0))

        # Step 2: Linearly transform the node features.
        # x = ...

        # Step 3: Compute the normalisation.
        # EXERCISE: Construct a size (E) array `norm` such that
        #   norm[i] = 1/sqrt(deg(edge_index[i,0]) * deg(edge_index[i,1])).
        norm = ... # This should be a 1D array of the same length as edge_index

        # Step 4: Call propagate. This calls self.message() on each edge,
        #   then aggregates (sums) messages at each target node.
        out = self.propagate(edge_index, x=x, norm=norm)

        return out + self.bias

    def message(self, x_j, norm):
        # EXERCISE: return the normalised message
        # (multiply each row of x_j by the corresponding entry of norm)
```

Compute the normalization

$$e = (i, j) \quad c_e = \frac{1}{\sqrt{\deg(i) \deg(j)}}$$

$$\text{norm} = E \begin{bmatrix} \square \\ \square \\ \square \\ \square \\ \square \end{bmatrix}$$

GCN in Pytorch Geometric

We will use **Pytorch Geometric**, a library built on top of Pytorch to work with GNNs.

```
class GCNConv(MessagePassing):
    def __init__(self, in_channels, out_channels):
        super().__init__(aggr='add') # "add" aggregation (sum over neighbours)
        self.lin = nn.Linear(in_channels, out_channels, bias=False)
        self.bias = nn.Parameter(torch.zeros(out_channels))

    def forward(self, x, edge_index):
        # Step 1: Add self-loops to the edge index.
        edge_index, _ = add_self_loops(edge_index, num_nodes=x.size(0))

        # Step 2: Linearly transform the node features.
        # x = ...

        # Step 3: Compute the normalisation.
        # EXERCISE: Construct a size (E) array `norm` such that
        #   norm[i] = 1/sqrt(deg(edge_index[i,0]) * deg(edge_index[i,1])).
        norm = ... # This should be a 1D array of the same length as edge_index

        # Step 4: Call propagate. This calls self.message() on each edge,
        #   then aggregates (sums) messages at each target node.
        out = self.propagate(edge_index, x=x, norm=norm)

        return out + self.bias

    def message(self, x_j, norm):
        # EXERCISE: return the normalised message
        # (multiply each row of x_j by the corresponding entry of norm)
```

Compute the message for each edge

$$e = (i, j) \mapsto \frac{1}{\sqrt{\deg(i) \deg(j)}} W x_j$$

$$\text{norm} = E \begin{bmatrix} \square \\ \square \\ \square \\ \square \\ \square \end{bmatrix} \quad x_j = E \begin{bmatrix} \square & \square & \square \\ \square & \square & \square \\ \square & \square & \square \\ \square & \square & \square \\ \square & \square & \square \end{bmatrix}$$

GCN in Pytorch Geometric

We will use **Pytorch Geometric**, a library built on top of Pytorch to work with GNNs.

```
class GCNConv(MessagePassing):
    def __init__(self, in_channels, out_channels):
        super().__init__(aggr='add') # "add" aggregation (sum over neighbours)
        self.lin = nn.Linear(in_channels, out_channels, bias=False)
        self.bias = nn.Parameter(torch.zeros(out_channels))

    def forward(self, x, edge_index):
        # Step 1: Add self-loops to the edge index.
        edge_index, _ = add_self_loops(edge_index, num_nodes=x.size(0))

        # Step 2: Linearly transform the node features.
        # x = ...

        # Step 3: Compute the normalisation.
        # EXERCISE: Construct a size (E) array `norm` such that
        #   norm[i] = 1/sqrt(deg(edge_index[i,0]) * deg(edge_index[i,1])).
        norm = ... # This should be a 1D array of the same length as edge_index

        # Step 4: Call propagate. This calls self.message() on each edge,
        #   then aggregates (sums) messages at each target node.
        out = self.propagate(edge_index, x=x, norm=norm)

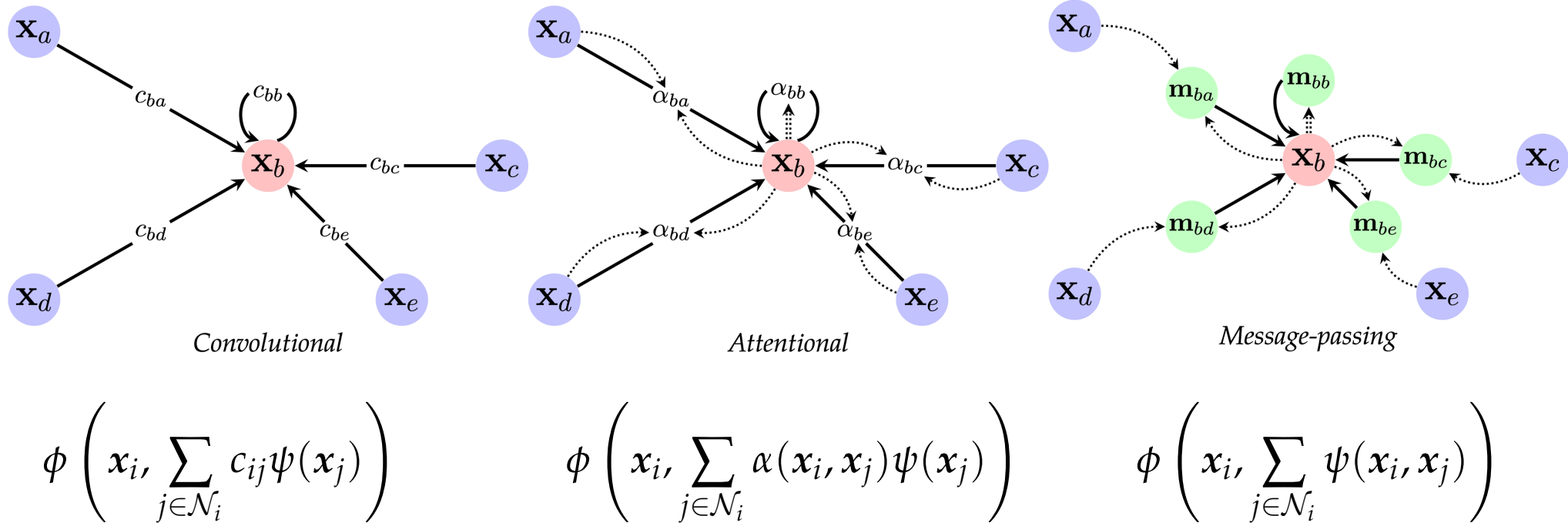
        return out + self.bias

    def message(self, x_j, norm):
        # EXERCISE: return the normalised message
        # (multiply each row of x_j by the corresponding entry of norm)
```

Use **propagate**, which applies the **message** function, and aggregates the messages at the target node.

Finally, add the bias, and return the output (The nonlinearity is applied when building the full network.)

Hierarchy of GNN Layers

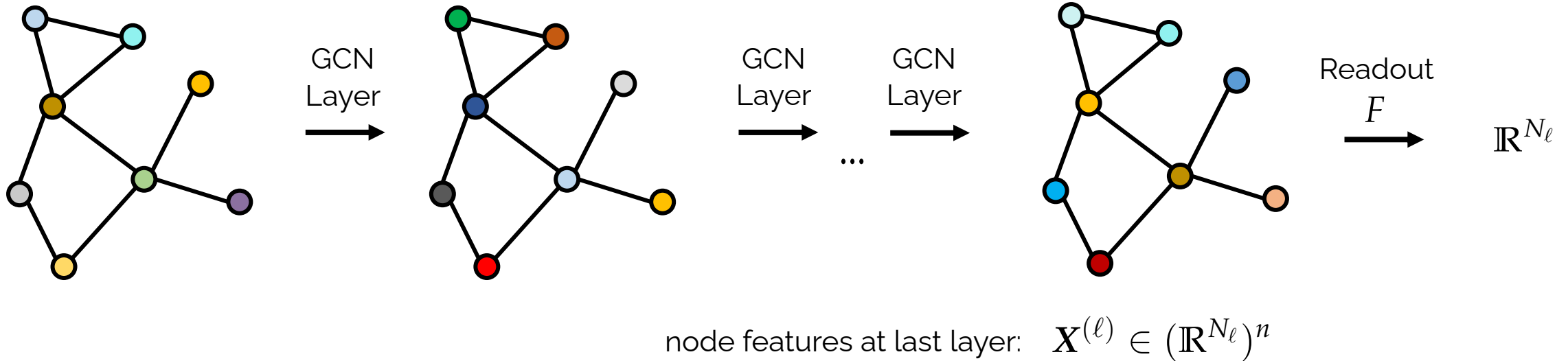


General Structure of Message-Passing GNNs:

1. For each edge, compute a **message** from source node to target node
2. For each node, **aggregate** message from its neighbours
3. For each node, **update** the feature

Graph Neural Networks

Graph Neural Networks (GNNs) are constructed by iterating GNN layers (such as GCN).



Node-Level Tasks:

Perform regression/classification on the node features at the last layer

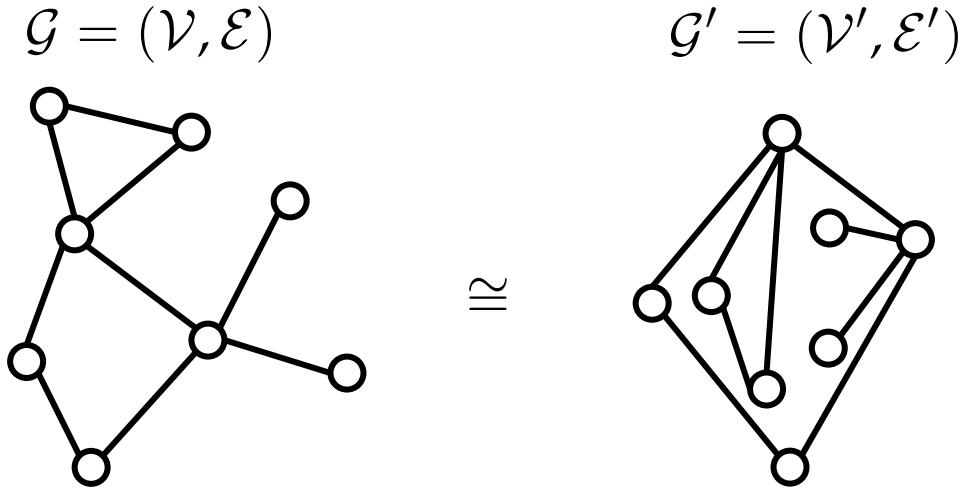
$$\mathbf{X}^{(\ell)} \in (\mathbb{R}^{N_\ell})^n$$

Graph-Level Tasks:

Perform regression/classification on the **readout** (this should be permutation **invariant**)

$$F(\mathbf{X}^{(\ell)}) = \sum_{i=1}^n x_i^{(\ell)}$$

How Powerful are GNNs?



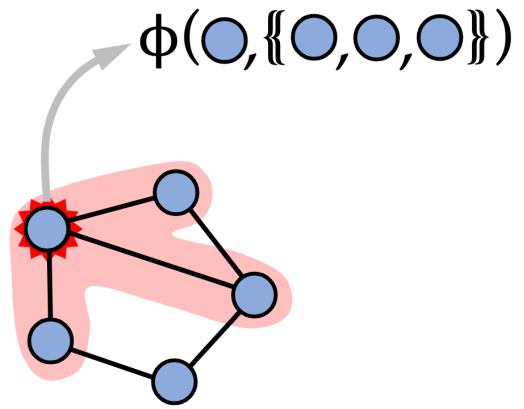
Two graphs are **isomorphic** if there exists an edge-preserving bijection

$$\phi : \mathcal{V} \rightarrow \mathcal{V}' \qquad i \sim j \iff \phi(i) \sim \phi(j)$$

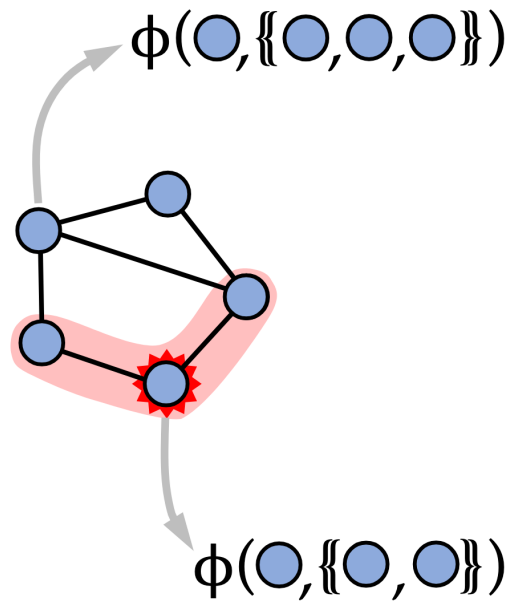
The complexity of graph isomorphism testing is an open problem (no known polynomial time algorithm, and not known if it's NP complete).

How can we quantify which graphs GNNs can distinguish?

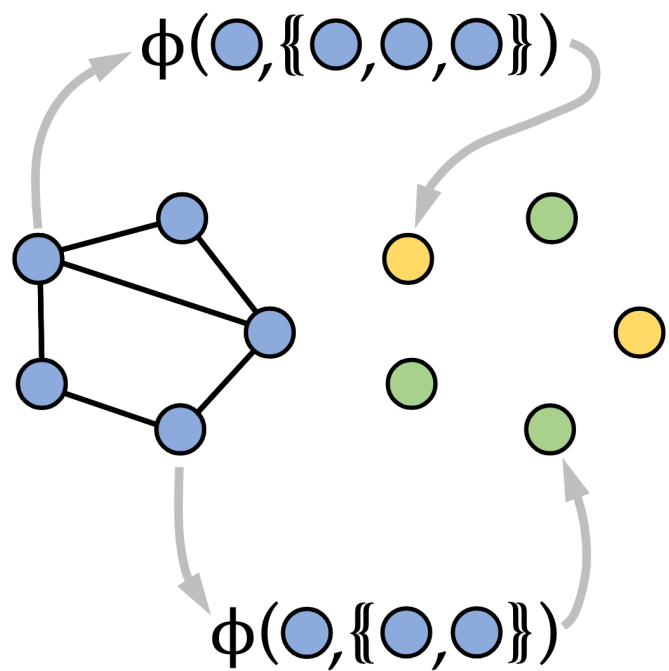
Weisfeiler-Lehman Test



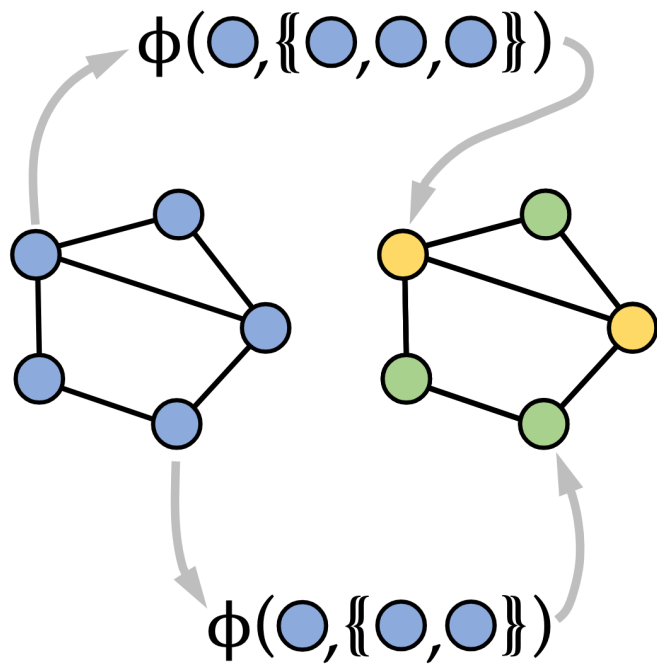
Weisfeiler-Lehman Test



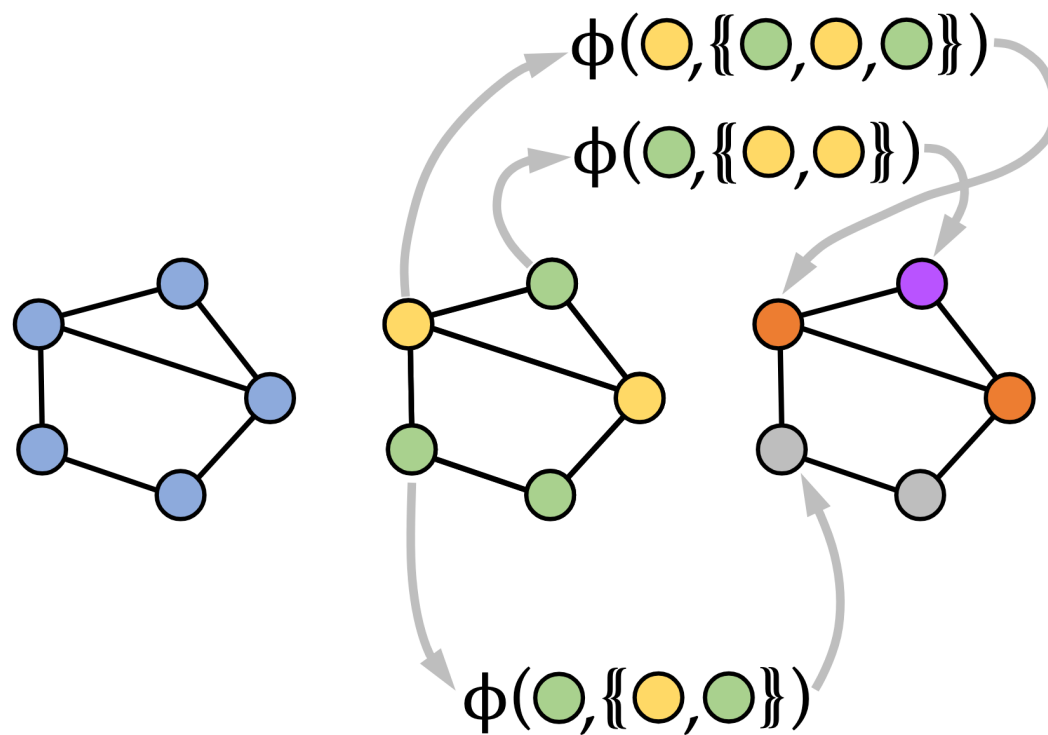
Weisfeiler-Lehman Test



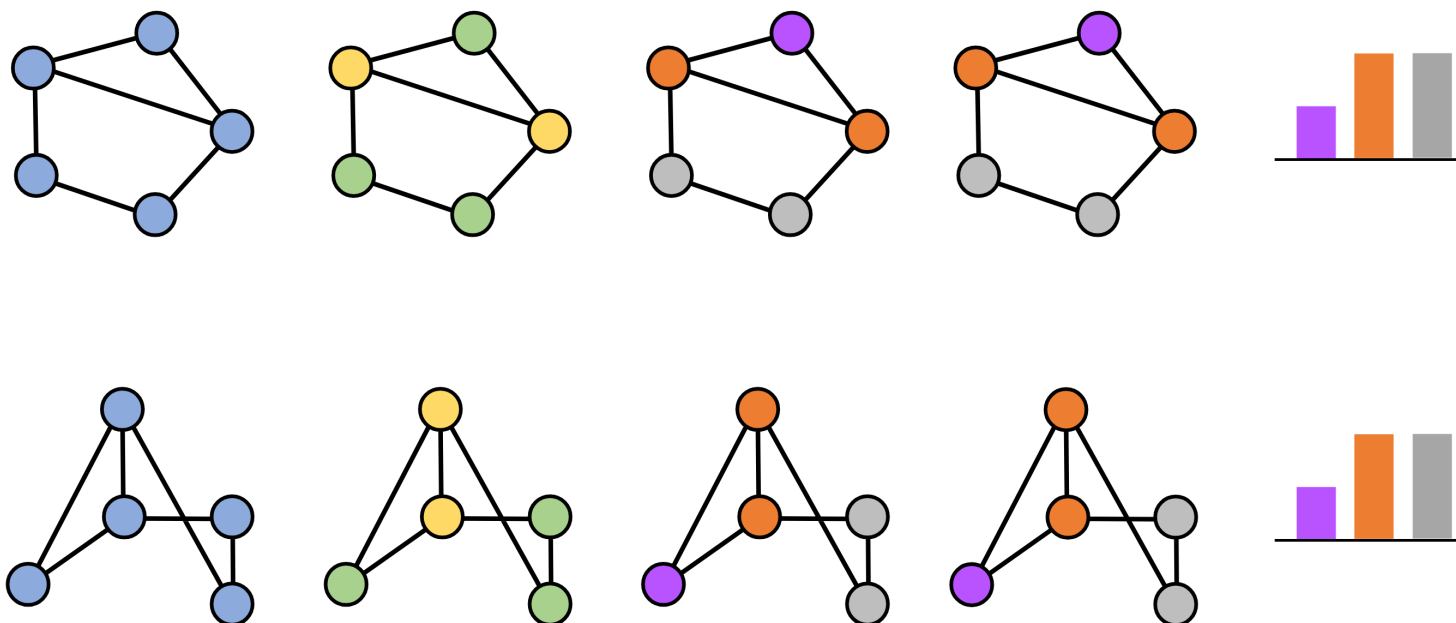
Weisfeiler-Lehman Test



Weisfeiler-Lehman Test



Weisfeiler-Lehman Test



For two graphs, if the histogram of “colors” is different at any iteration of the WL-test then we say that the two graphs are **distinguishable via the WL-test**.

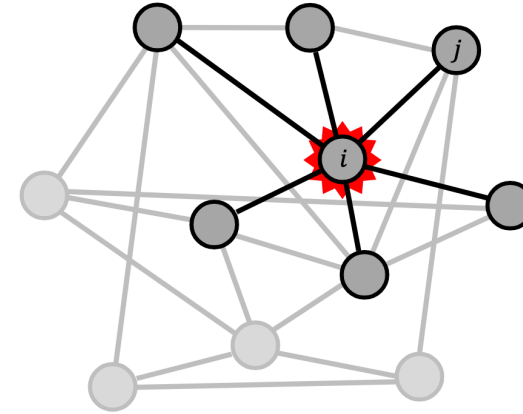
Weisfeiler-Lehman Test



These two graphs are **not** isomorphic, but the WL-test cannot distinguish between them.

Weisfeiler-Lehman Test

- **WL-test:** $\mathbf{x}_i \leftarrow \phi(\mathbf{x}_i, \{\{\mathbf{x}_j : j \in \mathcal{N}_i\}\})$
where ϕ is *injective* (hash function)
- **MPNN:** $\mathbf{x}_i \leftarrow \phi\left(\mathbf{x}_i, \bigoplus_{j \in \mathcal{N}_i} \psi(\mathbf{x}_i, \mathbf{x}_j)\right)$



Theorem [Xu et al. '19]

- 1) If an MPNN can distinguish two graphs, then the WL-test can distinguish between them.
- 2) There exists an MPNN such that if the WL-test can distinguish between them, then the MPNN can distinguish between them.

This result also holds when graphs are equipped with node features valued in a countable set.

Summary

GNNs are neural network architectures which use both features and relationships between nodes.

Workshop:

- 1) Working with graph data
- 2) Node classification with the Cora dataset
- 3) Graph regression with the ZINC dataset

Thanks!